

10) strchr retourne l'adresse de la première position d'un caractère dans une chaîne.

char ch[] = "Bonjour";

char\* adr = strchr(ch, 'a'); retourne l'adresse de la première occurrence du caractère 'a' dans la chaîne ch

Exemple: adr = strchr(ch, 'o');

printf("on aura la chaîne \"%s\"", adr);

Exécution: on aura la chaîne on

4°) skiphrk : retourne l'adresse du premier  
occurrence de l'un des caractères d'une  
chaîne ch2 dans une chaîne ch1

char ch1[] = "abc1def25zx";  
char ch2[] = "0123456789";

char\* skiphrk (ch1, ch2);

on aura donc addr : 1def25zx

Exercice : Écrire un programme qui remplace  
tous les chiffres d'une chaîne de caractères  
par des espaces.

5°) atoi : convertit une chaîne de caractères  
en un entier

Exemples a) atoi("125") → 125

b) atoi("12b") → 12

c) atoi("b12") → 0

a) ch → nb entier  
nb = 0

'1' nb = 0 \* 10 + 1 = 1

'2' nb = 1 \* 10 + 2 = 12

'5' nb = 12 \* 10 + 5 = 125

b) "12b"

nb = 0

'1' nb = 0 \* 10 + 1 = 1

'2' nb = 1 \* 10 + 2 = 12

'b' n'est pas un chiffre

c) "b"

nb = 0

'b' n'est pas un chiffre.

|   |    |      |      |    |      |
|---|----|------|------|----|------|
|   | ④  | ③    | ②    | ①  |      |
|   | 15 | 932  | 498  | 10 | 1275 |
| + |    | 1110 | 9815 | 3  | 4268 |
| = |    |      |      |    |      |

← ch1

← ch2

↑   ↑   ↑   ↑   ↑

```

int n;
printf("Entrez un entier : ");
scanf("%d", &n);
printf("n = %d", n);

```

Exemples :

1) Entrez un entier : (12)

$n = 12$

1) Entrez un entier : 12

$n = 12$

1) Entrez un entier : a)

$n = \text{value aléatoire}$

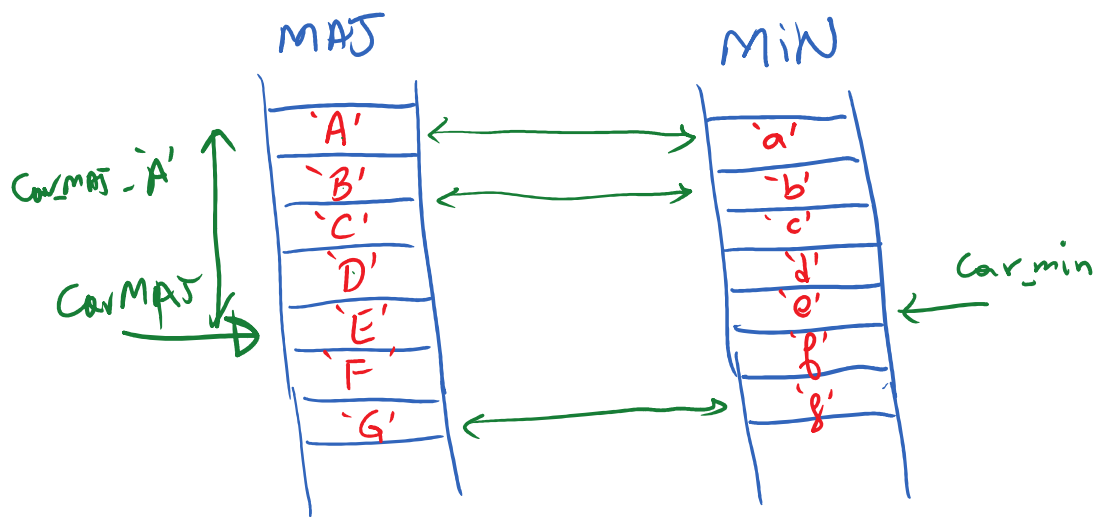
6) atof convertit une chaîne en float

Exemple :  $\text{atof}("125.75") \rightarrow 125.75$   
 $\text{atoi}("125.75") \rightarrow 125$

7) atol convertit une chaîne en long

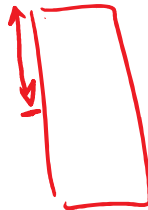
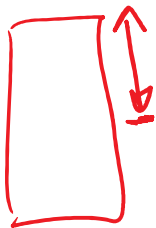
Exercice :

Exercice 1 : Écrire un programme qui permet de convertir une lettre minuscule en majuscule et l'inverse aussi.



$$\text{Car\_MAX} = \cancel{'A'} + \text{Car\_MAX} - \cancel{'A'}$$

$$\text{Car\_min} = \cancel{'a'} + \text{Car\_MAX} - \cancel{'A'}$$



$$\text{Car\_min} = \text{Car\_MAX} + 'a' - 'A';$$

$$= \text{Car\_MAX} + 97 - 65;$$

$$\text{Car\_min} = \text{Car\_MAX} + 32;$$

---

l'inverse :  $\text{Car\_MAX} = \text{Car\_min} - 32;$

```
#include <stdio.h>
```

```
main()
```

```
{ char ca[MAX], Car_min;
```

```
printf("Entrez une lettre majuscule : ");
scanf("%c", &Car_max);
```

```
if (Car_max > 'A' && Car_max <= 'Z')
```

```
{ Car_min = Car_max + 32;
```

```
printf("La lettre minuscule de %c est %c\n",
```

```
cor_max, cor_min);
```

```
    }  
    else  
    {  
    }
```

```
printf("Ce caractere n'est pas une lettre  
majuscule");
```

```
}
```

Exercice 2 : Ecrire un programme qui convertit tous  
les lettres minuscules d'une chaîne en des  
lettres majuscules

```
main()
```

```
{ char ch_max[30], ch_min[30]; int i;
```

```
printf("Entrez une chaîne de caractères:");
```

```
gets(ch_min);
```

```
    i=0;
```

```
    while (ch_min[i] != '\0')
```

```
    { if (ch_min[i] >='a' && ch_min[i] <='z')
```

```
        { ch_max[i] = ch_min[i] - 32;
```

```
        }  
        else
```

```
        { ch_max[i] = ch_min[i];
```

```
        }
```

```
        i++;
```

```
// for (i=0; ch_min[i] != '\0'; i++)
```

```
// {
```

```
//     }
```

```
printf("La chaîne majuscule de la chaîne  
minuscule %s est %s\n", ch_min, ch_max);
```

```
}
```

Ex ci dessus : Remplacer tous les chiffres par  
des espaces

1<sup>ere</sup> methode:

```
main()
```

```
{ char ch[30];
```

```
int i;
```

```
printf("Entrez une chaîne de caractères");
```

```
gets(ch);
```

```
printf("La chaîne passée est %s\n", ch);
```

```
for(i=0; ch[i]!='\0'; i++)
```

```
{ if (ch[i]=='0' || ch[i]=='1' || ch[i]=='2'
```

```
|| ch[i]=='3' || ch[i]=='4' || ch[i]=='5'
```

```
|| ..... || ch[i]=='9')
```

```
{ ch[i]=' ';
```

```
}
```

```
printf("La chaîne obtenue est %s\n", ch);
```

```
}
```

ou bien,

```
if (ch[i]>='0'
```

```
&& ch[i]<='9')
```

2<sup>eme</sup> methode :

```
#include <string.h>
```

```
main()
```

```

} char ch[30];
  char chiffre[11] = "0123456789";
  char *adr = ch;
  printf("Enter ch:");
  gets(ch);

  adr = strpbrk(adr, chiffre);
  while (adr != NULL)
  {
    *adr = ' ';
    adr = strpbrk(adr, chiffre);
  }
  printf("La chaine obtenue est \"%s\", ch);

```

Execution:

Enter ch: Mon 12 7m 4d ↵

- 1°) adr = ch.  
 adr = strpbrk(adr, chiffre) → 12 7m 4d  
 \*adr = ' ', → 12 7m 4d
- 2°) adr = strpbrk(adr, chiffre) → 2 7m 4d  
 \*adr = ' ', → 2 7m 4d
- 3°) adr = strpbrk(adr, chiffre) → 7m 4d.  
 \*adr = ' ', → 2 m 4d
- 4°) adr = strpbrk(adr, chiffre) → 4d  
 \*adr = ' ', → 2 d
- 5°) adr = strpbrk(adr, chiffre) → NULL

## Exercice de la semaine

a/

```
#include <stdio.h>
#include <string.h>
```

```
main()
```

```
{ char ch1[20], ch2[30], ch3[26], x[15];
```

```
int l1, l2;
```

```
printf("Entrez ch1:");
```

```
gets(ch1);
```

```
printf("Entrez ch2:");
```

```
gets(ch2);
```

```
l1 = strlen(ch1)/2; ←
```

```
l2 = strlen(ch2)/2; ←
```

```
strcpy(ch3, ch1, l1); ←
```

```
ch3[l1] = '\0'; ←
```

```
strcpy(x, ch2, l2); } ou bien  
strncat(ch3, x, l2);
```

```
ch3[l1+l2] = '\0'; ←
```

```
printf("La chaîne obtenue est \"%s\", ch3);
```

```
{
```

b/

```
main()
```

```
{
```

```
i = 0;
```

```
while (ch1[i] != '\0') i++;
```

```
l1 = i/2;
```

```
i = 0;
```

```
while (ch2[i] != '\0') i++;
```

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| ↑ | ↑ | ↑ | ↑ | ↑ | ↑ |

i = 5



```

    l2 = i / 2;
    for (i = 0; i < l1, i++)
        ch3[i] = ch1[i];
    for (i = 0; i < l2, i++)
        ch3[l1 + i] = ch2[i];
    ch3[l1 + l2] = '\0';
    printf("La chaîne obtenue est %s\n", ch3);
}

```

Rappel : / pour passer un tableau d'entiers par exemple  
à ne fait.

```

type_retour nom_fct (int *t, int n)
{
    ~
}

```

1) pour passer un tableau de réels

```

type_retour nom_fct (float *t, int n)
{
    ~
}

```

2) pour passer un tableau de caractères

```

type_retour nom_fct (char *t)
{
    ~
}

```

0.) Un tableau entier  $\text{int } t[50];$   
 Un tableau de reels  $\text{float } t[50];$   
 / / / variables  $\text{char } t[100];$   
 Un tableau de chaines de caracteres

char t[50][15];

Rappel sur les tableaux à deux dimensions.  
ou bien une matrice.

$X = M[0]$   
 $Y = M[1]$   
 $Z = M[2]$   
 $U = M[3]$

|                 |                 |                 |
|-----------------|-----------------|-----------------|
| $M[0][0]$<br>3  | $M[0][1]$<br>-7 | $M[0][2]$<br>10 |
| $M[1][0]$<br>-1 | $M[1][1]$<br>4  | $M[1][2]$<br>5  |
| $M[2][0]$<br>7  | $M[2][1]$<br>2  | $M[2][2]$<br>9  |
| $U[0]$<br>0     | $U[1]$<br>6     | $U[2]$<br>12    |

Exemples: `int M[4][3];`

Chaque elt de  $M$  est sous forme  $M[i][j]$

ou i de o a 3  
j de o a 2

pour traiter les elts de la matrice M

```
for(i=0; i < NL; i++)  
{  
    for(j=0; j < NC; j++)  
        // M[i][j]  
}
```

lorsque  $NL = NC$  on parle d'une matrice carrée.

La matrice :

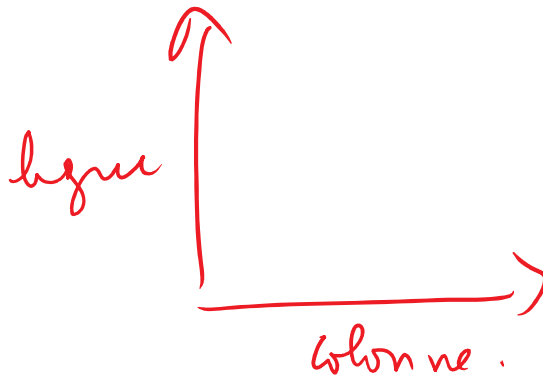
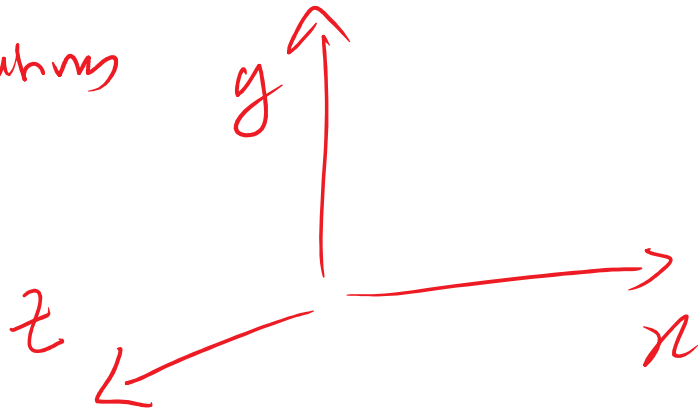


tableau à trois dimensions



```
float Note[DimEtud][DimMat][DimNot];
```

```
for(i=0; i < ; i++)
```

```
{ for(j=0; j < ; j++)
```

```
{ for(k=0; k < ; k++)
```

```
{ ~ M[i][j][k] ~
```

```
{
```

```
}
```

Si un tableau de chaînes de caractères (mot)

```
char mot[10][15];
```

on aura besoin d'une  
seule boucle car un  
mot on peut le pointer  
en entier

Exercice :

Écrire un prog qui saisit exactement une phrase  
de cinq mots puis l'affiche comme suit :

Exemple : Entrez la phrase :  
Bonjour tout le monde !  
! monde le tout Bonjour

## Voit le programme phrase enver

Exercice : Écrire un programme qui demande  
un verbe du premier groupe puis  
le conjugue au présent de l'indicatif

l  
Chanter :  
↑  
10

|       |          |
|-------|----------|
| je    | chante   |
| tu    | chantes  |
| il    | chante   |
| elle  | chante   |
| nous  | chantons |
| vous  | chantez  |
| ils   | chantent |
| elles | chantent |

~~~~~  
pronoms                      suffixes

voit conjugaison (char & verb)

{ char pronoms [8][6] = } —

char suffixes [8][4] = } —

int l, i;

l = strlen(verb);

~~~~~

## Exn 2 Levenshtein

10) char txt[100] = "bunp";

Diagram illustrating the character array `txt` (size 100) containing the string "bunp". The array is shown with indices 0 to 9. The characters are 'b', 'u', 'n', 'p', followed by null terminators. Red arrows indicate the current position `i` (pointing to the first null terminator) and the previous position `i-1` (pointing to the last character 'p'). The calculation `i-1-0+1` is shown below the array.

20) char prefix[20] = "bun";

if ( strcmp(txt, prefix, strlen(prefix)) == 0 )  
 return 1;  
 else  
 return 0;

30) char suffix[20] = "p";

$l1 = \text{strlen}(txt);$   
 $l2 = \text{strlen}(suffix);$   
 $\text{strcmp}(txt + l2 - l1, suffix, l2)$

```
int longueur(char* txt)
```

```
{  
    int i = 0;  
    while (txt[i] != '\0')  
        i++;  
    return i;  
}
```

```
int prefixe(char* txt, char* pref)
```

```
{  
    int l;
```

```
    l = strlen(pref);
```

```
    if (strncmp(txt, pref, l) == 0) {  
        return 1;  
    }  
    else {  
        return 0;  
    }  
}
```

ou bien

```
return  
(!strncmp(txt, pref, l));
```

```
int suffixe(char* txt, char* suf)
```

```
{  
    int l1, l2;
```

```
    l1 = strlen(txt);
```

```
    l2 = strlen(suf);
```

```
    if (strncmp(txt + (l1 - l2), suf, l2) == 0)  
        return 1;  
}
```

```
else  
    return 0;  
}
```

```
main()  
{ char chaine[30], ch_pref[15], ch_suf[10];  
  int l, p;  
  printf("Entrez votre chaine: ");  
  gets(chaine);  
  printf("Entrez votre prefixe: ");  
  gets(ch_pref);  
  printf("Entrez votre suffixe: ");  
  gets(ch_suf);  
  l = longueur(chaine);  
  printf("La longueur de la chaine %s est  
        %d\n", chaine, l);  
  p = prefixe(chaine, ch_pref);  
  if (p == 1)
```



printf(" la chaine %s est un prefixe  
de la chaine '%s'\n", ch\_pref, chain);

else  
printf(" n'est pas un prefixe\n");

p = suffixe(chain, ch\_suf);

if (p == 1)

else

{